

BM25 알고리즘 이해와 구현

TF-IDF의 한계를 넘어선 확률적 검색 모델

TF-IDF 기초 → 두 가지 한계 → BM25 수식 → 파라미터 튜닝 → 실무 비교

TF (Term Frequency) — 용어 빈도

- 해당 문서에서 특정 단어가 몇 번 나오는가

$$TF(t, d) = f(t, d) / |d|$$

- $f(t, d)$: 문서 d 에서 단어 t 등장 횟수
- $|d|$: 문서 d 의 전체 단어 수

예: 100단어 문서, "Python" 5번 등장

$$TF = 5/100 = 0.05$$

IDF (Inverse Document Frequency) — 역문서 빈도

- 전체 컬렉션에서 해당 단어가 얼마나 희귀한가

$$IDF(t) = \log(N / df(t))$$

- N : 전체 문서 수
- $df(t)$: 단어 t 가 등장하는 문서 수

예: 1,000문서 중 "Python" 50곳 등장

$$IDF = \log(1000/50) = 3.0$$

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

두 값의 곱 → 이 문서에서 이 단어가 얼마나 '독특하게 중요한가'

💡 흔한 단어("은/는")는 $IDF \downarrow \rightarrow TF-IDF \downarrow$ / 희귀 키워드("BM25")는 $IDF \uparrow \rightarrow TF-IDF \uparrow$

TF	IDF	TF-IDF 점수	해석 / 예시
높음	낮음	중간	"은/는" 같은 조사 — 흔하지만 이 문서엔 많음
높음	높음	높음 ★	이 문서의 핵심 키워드 — 희귀하고 많이 나옴
낮음	높음	중간	부수적 언급 — 희귀하지만 한두 번만 나옴
낮음	낮음	낮음 ✕	의미 없는 단어 — 흔하고 이 문서에도 적게 나옴

TF-IDF의 직관: 문서 내에서 '독특하게 자주 나오는 단어'를 핵심 키워드로 판별 — 검색어 매칭의 기반

TF-IDF의 두 가지 한계 — BM25가 필요한 이유

이론

문제 1 — TF 포화 없음

단어가 10번 나오든 100번 나오든 TF는 선형으로 계속 증가

하지만 실제로는:

5번 나온 문서 = 50번 나온 문서와 비슷하게 관련 있음

TF-IDF에서 "Python" 5번 → $TF=0.05$

TF-IDF에서 "Python" 50번 → $TF=0.50$ (10배!)

실제 관련성 차이는 거의 없음

문제 2 — 문서 길이 미보정

1,000단어 문서에서 "Python" 5번

100단어 문서에서 "Python" 5번

→ 의미가 완전히 다름!

1,000단어 → $TF=0.005$ (밀도 낮음)

100단어 → $TF=0.050$ (밀도 높음)

TF-IDF는 이 차이를 충분히 반영하지 못함

BM25의 해결책

k_1 파라미터 — TF가 일정 수준 이상 올라가도 더 이상 선형으로 증가하지 않게 포화(saturation) 적용

b 파라미터 — 문서 길이를 컬렉션 평균($avgdl$)과 비교해 정규화, 긴 문서 불이익·짧은 문서 이익 방지

$$\text{BM25}(q,d) = \sum_{t \in q} \text{IDF}(t) \times \left[\frac{f(t,d) \times (k_1+1)}{f(t,d) + k_1 \times (1 - b + b \times |d|/\text{avgdl})} \right]$$

$t \in q$

t : 쿼리 내 각 단어 | $f(t,d)$: 문서 d 에서 단어 t 빈도 | $|d|$: 문서 길이 | avgdl : 평균 문서 길이

① IDF 항

$\text{IDF}(t)$

- TF-IDF의 IDF와 동일한 역할
- 희귀한 단어일수록 가중치 높음
- BM25의 IDF는 평활화 적용:
- $\log((N-\text{df}+0.5) / (\text{df}+0.5) + 1)$

② TF 포화 항 (k_1)

$$\frac{f(t,d) \times (k_1+1)}{f(t,d) + k_1 \times \dots}$$

- 분자 (k_1+1), 분모 k_1 으로 상한 생성
- TF가 아무리 커도 점수 포화
- $k_1=1.2$ 기준: $\text{tf}=5$ 와 $\text{tf}=50$ 점수 유사
- → 단어 폭탄 방지

③ 길이 정규화 항 (b)

$$1 - b + b \times |d|/\text{avgdl}$$

- 문서가 평균보다 길면 분모 증가 → 점수↓
- 문서가 평균보다 짧으면 분모 감소 → 점수↑
- $b=0$: 길이 무시 / $b=1$: 완전 정규화
- $b=0.75$: 실무 표준

RAG 평가 프레임워크

Precision & Recall



TP / FP / FN / TN



Precision@K & Recall@K



Trade-off 관계



R-Precision



F1@K & F-beta



실무 적용 시나리오

이 강의에서 배울 내용

01

TP / FP / FN / TN

4가지 판정 케이스와 혼동 행렬

02

Precision@K & Recall@K

수식, 직관, 계산 예시

03

Trade-off 관계

K 변화에 따른 반비례 구조

04

R-Precision

공정한 비교를 위한 특수 케이스

05

F1@K & F-beta

조화평균 & 가중 조화평균

06

코드 구현 & 실무 시나리오

Python 코드 + RAG 적용 기준

TP · FP · FN · TN

4가지 판정 케이스

코로나 양성

TP

True Positive

맞게 포착 ✓

FP

False Positive

노이즈 ✗

FN

False Negative

누락 ✗

TN

True Negative

올바른 제외 ✓

코로나 환자

코로나 아님

4가지 판정 케이스

코로나 양성

TP

True Positive

맞게 포착 ✓

FP

False Positive

노이즈 ✗

FN

False Negative

누락 ✗

TN

True Negative

올바른 제외 ✓

코로나 환자

코로나 아님

Precision · Recall · F1 수식

Precision

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

양성 진단자 중 실제 환자 비율

$\text{FP} \uparrow \rightarrow \text{Precision} \downarrow$

Recall

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

실제 환자 중 잡아낸 비율

$\text{FN} \uparrow \rightarrow \text{Recall} \downarrow$

F1

$$\text{F1} = 2 \times \text{P} \times \text{R} / (\text{P} + \text{R})$$

조화평균
둘 다 높아야 ↑

하나라도 0 $\rightarrow \text{F1} = 0$

검색 결과의 4가지 판정 케이스

TP

True Positive

관련 있고
검색도 됨

맞게 포착 ✓

FP

False Positive

무관한데
검색에 나옴

노이즈 ✗

FN

False Negative

관련 있는데
검색 못 함

누락 ✗

TN

True Negative

무관하고
검색도 안 됨

올바른 제외 ✓

검색됨

TN은 왜
무관한가?

미검색 무관 문서는 수
백만 개. 의미 없음.

실제 관련

실제 무관

Precision · Recall · F1 수식

Precision

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

검색된 것 중
진짜 관련인 비율

$\text{FP} \uparrow \rightarrow \text{Precision} \downarrow$

Recall

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

전체 관련 문서 중
잡아낸 비율

$\text{FN} \uparrow \rightarrow \text{Recall} \downarrow$

F1

$$\text{F1} = 2 \times \text{P} \times \text{R} / (\text{P} + \text{R})$$

조화평균
둘 다 높아야 ↑

하나라도 0 $\rightarrow \text{F1} = 0$

Precision@K & Recall@K

Precision@K — 보여준 것 중 맞은 비율

$$\text{Precision@K} = (\text{상위 K개 중 관련 문서 수}) / K$$

K	검색 결과 (● = 관련)	관련 수	P@K
K=1	●	1	1.000
K=3	● ○ ●	2	0.667
K=5	● ○ ● ○ ●	3	0.600
K=10	● ○ ● ○ ● ○ ○ ○ ○ ○	3	0.300

💡 K가 커질수록 관련 없는 문서가 섞여 들어와 Precision은 하락하는 경향입니다.

Recall@K — 전체 관련 문서 중 찾아낸 비율

$$\text{Recall@K} = (\text{상위 K개 중 관련 문서 수}) / \text{전체 관련 문서 수(R)}$$

K	검색 결과 (총 관련 5개)	찾은 수/R	R@K
K=1	●	1/5	 0.200
K=3	● ○ ●	2/5	 0.400
K=5	● ○ ● ○ ●	3/5	 0.600
K=10	● ○ ● ○ ● ○ ○ ○ ○ ○	3/5	 0.600

💡 K가 커질수록 관련 문서를 더 많이 포함하므로 Recall은 상승하는 경향입니다.

Precision@K vs Recall@K — 핵심 차이

구분	Precision@K	Recall@K
분모	K (검색 결과 수)	R (전체 관련 문서 수)
핵심 질문	"보여준 것 중 쓸모있는 건 몇 %?"	"있는 것 중 못 찾은 건 없나?"
K↑ 경향	하락 (노이즈 유입)	상승 (더 많이 커버)
모르는 것	놓친 관련 문서 존재 여부	결과에 노이즈가 얼마나 섞였는지

⚠ Precision 낮으면

LLM에 노이즈 청크 전달 → 환각(Hallucination) 유발

⚠ Recall 낮으면

핵심 정보 누락 → 불완전한 답변 생성

Trade-off 관계

왜 Trade-off가 발생하는가?

검색 시스템은 관련도 점수(relevance score) 기준으로 문서를 정렬한 뒤, 임계값 이상의 문서만 반환합니다.

▲ 임계값

★★★★★

★★★★☆

★★★★☆

★★★☆☆

★★★☆☆

★★★☆☆

임계값 높이면 (엄격)

- 반환 문서 수 ↓
- 가져온 것들은 거의 정답 → Precision ↑
- 정답 문서를 일부 놓침 → Recall ↓

임계값 낮추면 (느슨)

- 반환 문서 수 ↑
- 정답을 빠짐없이 포함 → Recall ↑
- 오답 문서도 섞여 들어옴 → Precision ↓

K 변화에 따른 Precision↓ & Recall↑

K ↑ → Precision ↓

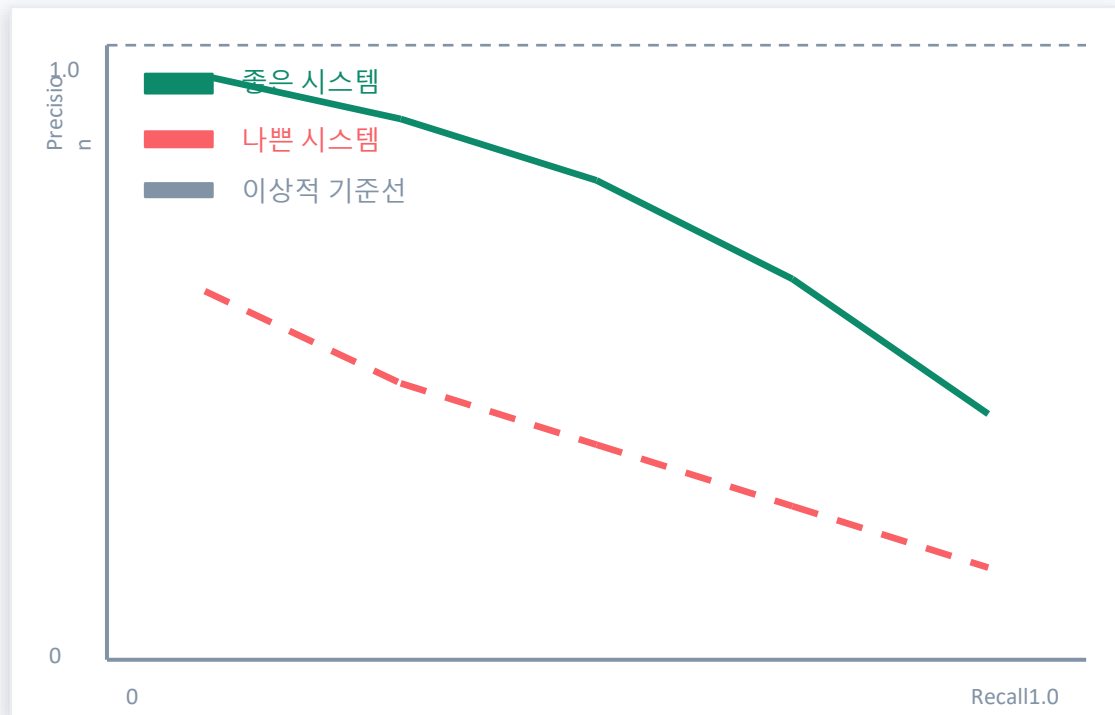
K ↑ → Recall ↑

K	반환 문서	그 중 정답	P@K	R@K
3	3개	3개	1.00	0.60
5	5개	4개	0.80	0.80
10	10개	5개	0.50	1.00

🐟 비유: 그물코 촘촘(K↑) → 작은 물고기까지 + 돌맹이도 딸려옴 → Recall↑, Precision↓
그물코 크게(K↓) → 큰 물고기만 선별, 작은 물고기 빠져나감 → Precision↑, Recall↓

Precision-Recall 곡선 (PR Curve)

K 또는 임계값을 연속적으로 변화시키며 (Recall, Precision) 점을 이어 PR 곡선을 만듭니다.



곡선이 (1,1)에 가까울수록

좋은 시스템

이상적 시스템

Recall=1 & Precision=1

AP (Average Precision)

**PR 곡선 아래 면적
= 종합 성능 지표**

R-Precision

R-Precision — 공정한 비교를 위한 메트릭

$$\text{R-Precision} = \text{상위 } R \text{ 개 중 관련 문서 수} / R$$

R = 전체 관련 문서 수

✗ Precision@K의 한계

K를 임의로 정해야 함. 쿼리마다 관련 문서 수가 다르면 불공평한 비교.

☑ R-Precision의 장점

각 쿼리의 관련 문서 수 R을 자동으로 K로 사용 → "같은 조건"에서 공정하게 비교.

흥미로운 성질 — R-Precision에서 Precision = Recall

$$\text{R-Precision} = \text{TP} / R = \text{TP} / (\text{TP} + \text{FN}) = \text{Recall@R}$$

수식적으로는 같지만 - 분모를 K로 보는가, R로 보는가의 관점 차이

📖 비유: "10문제 중 8개 맞혔네" = Recall 느낌 | "딱 10문제만 풀었는데 8개 맞았네" = Precision 느낌

F1@K & F-beta

F1@K — Precision과 Recall의 조화평균

$$F1@K = 2 \times P@K \times R@K / (P@K + R@K)$$

왜 조화평균인가? 둘 중 하나라도 낮으면 전체 점수가 크게 떨어지도록 패널티 부여

P@K	R@K	산술평균	F1@K (조화평균)	차이
1.0	0.1	0.55	0.18	큰 패널티 ⚠
0.7	0.7	0.70	0.70	균형 ✓
0.8	0.6	0.70	0.686	미세 하락
⚡ P@K와 R@K 중 하나라도 0이면 F1@K = 0				

🎯 둘 다 높아야만 점수가 높게 나옴 — "균형"이 핵심

F-beta — β 로 Precision/Recall 중요도 조절

$$F_{\beta} = (1 + \beta^2) \times P \times R / (\beta^2 \times P + R)$$

$\beta < 1 \rightarrow$ Precision 중시 | $\beta = 1 \rightarrow$ 동일 (= F1) | $\beta > 1 \rightarrow$ Recall 중시

β 값	의미	주요 사용 시나리오	이유
$\beta = 0.5$	Precision 2배 가중	RAG 컨텍스트, 스팸 필터	노이즈 문서 $\rightarrow$ LLM 품질 저하
$\beta = 1.0$	동일 가중 (= F1)	일반 웹 검색, 추천 시스템	누락·오검색 동등하게 나쁨
$\beta = 2.0$	Recall 2배 가중	의료·법률 문서 검색, 실종자 수색	누락 = 치명적 결과
예시: P=0.9, R=0.4일 때 $\rightarrow F_{0.5} \approx 0.72$ $F_1 \approx 0.55$ $F_2 \approx 0.45$ (β 높을수록 낮은 Recall에 엄격)			

실무 적용 — 언제 어떤 메트릭을 쓸까?

RAG 컨텍스트 삽입

Precision 우선 ($\beta < 1$)

노이즈 문서가 LLM에 들어가면 답변 품질이 즉시 저하됩니다.
잘못 넣는 것이 못 넣는 것보다 더 해롭습니다.

의료·법률 문서 검색

Recall 우선 ($\beta > 1$)

관련 문서를 단 하나라도 놓치는 것이 치명적입니다.
약간의 노이즈는 감수할 수 있습니다.

일반 웹 검색·추천

F1 균형 ($\beta = 1$)

누락과 오검색 모두 사용자 경험을 해칩니다.
어느 한쪽이 더 나쁘다고 단정하기 어렵습니다.

스팸 필터

Precision 우선 ($\beta < 1$)

정상 메일을 스팸으로 잘못 분류하는 비용이 매우 큼니다.
스팸이 일부 통과하는 것보다 훨씬 심각한 문제입니다.

💡 선택 기준: 놓치는 것이 더 위험 $\rightarrow \beta \uparrow$ | 잘못 가져오는 것이 더 위험 $\rightarrow \beta \downarrow$ | 동등 $\rightarrow \beta = 1$

검색 메트릭 종합 비교

메트릭	범위	핵심 질문	강점	약점 / 한계
Precision@K	0-1	"보여준 k개 중 맞은 비율?"	직관적, 빠른 계산	관련 문서 총 수 무시
Recall@K	0-1	"전체 정답 중 찾아낸 비율?"	누락 측정 가능	노이즈 반영 못 함
R-Precision	0-1	"R개 중 맞은 비율 (공정 비교)"	쿼리 간 공정 비교	단일 임계점만 봄
F1@K	0-1	"Precision·Recall 균형?"	균형 평가	둘의 중요도 고정
F-beta	0-1	"어느 쪽이 더 중요한가?"	도메인별 가중치 조절 가능	β 설정이 주관적

실무 팁: 단일 메트릭으로 결론 짓지 말 것. Hit Rate + MRR + NDCG + F-beta를 함께 모니터링하는 것이 베스트 프랙티스.